

Введение

Структуры данных являются одной из базовых частей программирования. Они определяют, как данные хранятся в памяти, как быстро к ним можно получить доступ, как выполняются вставка, удаление, поиск и другие операции. Поэтому выбор структуры данных напрямую влияет на производительность программы.

Обычно эффективность структур данных оценивается с помощью асимптотической сложности. Например, для хеш-таблицы операции вставки, поиска и удаления в среднем имеют сложность $O(1)$, а для бинарной кучи операции вставки и удаления корня выполняются за $O(\log N)$. Такие оценки позволяют понять, как алгоритм ведет себя при росте размера входных данных.

Однако на практике одной только асимптотической сложности недостаточно. Две реализации одной и той же структуры данных могут иметь одинаковую теоретическую сложность, но при этом сильно отличаться по реальному времени работы. Причины таких различий связаны с постоянными множителями, языком программирования, устройством памяти, кэш-локальностью, количеством аллокаций, накладными расходами вызовов функций и внутренними оптимизациями библиотечных реализаций.

Например, собственная реализация хеш-таблицы на Python и встроенный `dict` имеют близкую ожидаемую сложность основных операций. Тем не менее `dict` часто оказывается значительно быстрее, потому что он реализован на низком уровне и содержит множество оптимизаций. Аналогично, собственная реализация бинарной кучи и стандартные библиотечные реализации могут иметь одинаковые асимптотические оценки, но различаться по постоянным факторам и особенностям внутренней реализации.

В данной работе рассматриваются собственные реализации структур данных на Python и C++, а также проводится их сравнение со встроенными и библиотечными аналогами. Основное внимание уделяется двум структурам данных: hash table и binary heap.

Hash table используется для хранения пар ключ--значение и обеспечивает быстрый доступ к элементам по ключу. В среднем операции вставки, поиска и удаления выполняются за $O(1)$. При этом практическая производительность хеш-таблицы сильно зависит от способа разрешения коллизий, коэффициента заполнения, стратегии расширения массива и качества организации памяти.

Binary heap используется как основа для реализации priority queue. В данной работе рассматривается min-heap, в котором минимальный элемент всегда находится в корне. Такая структура позволяет получать минимальный элемент за $O(1)$, а вставку и удаление корня выполнять за $O(\log N)$. На практике скорость работы кучи зависит не только от высоты дерева, но и от порядка входных данных, стоимости операций `sift up` и `sift down`, а также от эффективности работы с массивом.

Сравнение Python и C++ является важной частью работы, потому что эти языки имеют разные модели выполнения. Python является высокоуровневым языком с динамической типизацией, поэтому каждая операция содержит дополнительные накладные расходы. C++ компилируется в машинный код и дает больше контроля над размещением данных в памяти. За счет этого C++-реализации часто имеют меньшие постоянные расходы и лучше используют аппаратные возможности процессора.

При этом сравнение не сводится только к противопоставлению Python и C++. Важную роль играют и библиотеч-

ные реализации. Многие стандартные структуры данных реализованы значительно более оптимизированно, чем простые пользовательские реализации. Поэтому в работе сравниваются не только языки программирования, но и разные уровни реализации: собственный код, встроенные структуры и стандартные библиотечные контейнеры.

Актуальность работы заключается в том, что на практике разработчику важно понимать не только теоретическую сложность структуры данных, но и реальные факторы, влияющие на производительность. Одинаковая асимптотика не гарантирует одинакового времени работы. Поэтому экспериментальное сравнение помогает увидеть, какие различия возникают из-за алгоритма, какие — из-за языка программирования, а какие — из-за особенностей конкретной реализации.

Объектом исследования являются структуры данных и их практическая производительность при выполнении базовых операций.

Предметом исследования являются различия во времени выполнения операций собственных реализаций структур данных на Python и C++ по сравнению со встроенными и библиотечными аналогами.

Целью работы является исследование того, как особенности языка программирования, реализации и организации памяти влияют на практическую производительность структур данных при одинаковых или близких асимптотических оценках.

Для достижения поставленной цели необходимо решить следующие задачи:

1. рассмотреть теоретические свойства выбранных структур данных;
2. реализовать hash table и binary heap на Python и C++;
3. подобрать встроенные и библиотечные аналоги для сравнения;
4. разработать набор benchmark-сценариев для основных операций;
5. провести измерения на различных размерах входных данных;
6. выполнить несколько повторных запусков для каждого эксперимента;
7. вычислить статистические характеристики результатов, включая среднее значение, медиану и стандартное отклонение;
8. представить результаты в виде таблиц и графиков;
9. проанализировать причины различий производительности между реализациями;
10. сопоставить экспериментальные результаты с теоретическими оценками сложности.

Таким образом, работа направлена не только на реализацию структур данных, но и на исследование их поведения в реальных условиях. Основной акцент делается на том, почему структуры с одинаковыми теоретическими оценками могут показывать разную практическую производительность.